CS 101 - Hash Tables

Week 7. Lecture slides for the mid-term unit on dictionaries.

Instructor: Dr. Helen Park. These slides are the only reading for Friday.

Today: hash functions, collisions, chaining, open addressing, load factor.

Agenda

1. Why arrays fail as dictionaries when keys are strings.
2. A hash function maps a key to a bucket index.
3. Collisions are normal. We will compare chaining and open addressing.
4. Load factor decides when to grow the table.
5. We end with a worked example. Extra reading sits in the appendix.

Why not a sorted array?

Lookup in a sorted array of n pairs is binary search: O(log n). Insert still costs O(n) because you must shift.

A hash table targets expected O(1) lookup and insert, if the hash spreads keys and the table stays below a load threshold.

The cost is a worse worst case, and you give up iteration in key order unless you keep a second structure.

Hash functions

A hash function h takes a key and returns an integer. We then take h(key) mod m, where m is the number of buckets.

A good h looks random, is fast, and uses the whole key. Adding the characters of a string and stopping early is a bad hash: anagrams collide, and long keys that share a prefix cluster.

In this course we treat the hash as a black box that is uniform enough. You will not invent a cryptographic hash. You will pick a well-known string hash and test it on real words.

Collisions

A collision is two keys with the same bucket index. They are not a bug. The birthday bound says collisions appear well before n reaches m.

If you ignore collisions, the second key overwrites the first and you have a silent data loss. Every real table has a collision strategy.

Two strategies in this unit: chaining (a list per bucket) and open addressing (probe for the next empty slot).

Chaining

Each bucket holds a linked list of (key, value) pairs. Insert prepends. Lookup walks the list and compares keys, not hashes.

Expected list length is the load factor n/m. If h is uniform, lookup is expected O(1 + n/m).

Memory cost is one pointer per entry plus the table of m list heads. Deletion is straightforward: unlink the node.

Open addressing

All pairs live in the table itself. On a collision, probe: linear (i, i+1, i+2, ...) or quadratic.

Linear probing is simple and cache-friendly, and it forms runs of filled slots called clusters. A cluster makes later probes longer.

Deletion cannot leave a hole. You mark a slot as tombstone so a later lookup keeps probing. Tombstones slow the table until you rehash.

Load factor

Load factor alpha = n / m. For chaining, alpha may exceed 1. For open addressing, alpha must stay below 1, and performance falls apart above about 0.7.

When alpha crosses the threshold, allocate a larger table (usually double) and re-insert every live pair. That rehash is O(n), paid rarely, so amortized insert stays O(1) if you grow geometrically.

Never pick m as a power of two if your hash is just the low bits of an integer. A prime m is the usual fix in this course.

Worked example

m = 8 buckets. Keys: ant, bee, cat, dog, eel, fox. Use h(k) = sum of char codes mod 8.

You will see ant and cat share a bucket. With chaining they sit on one list. With linear probing, cat takes the next empty slot.

Draw both pictures before Friday. The exam will ask you to finish a diagram like this one, including one collision.

Mistakes I will mark

Comparing hashes instead of keys on lookup. Two different keys can share a hash.

Forgetting to rehash when alpha is high, then calling the table O(1) on a demo of ten keys.

Using a tombstone as empty during insert, which can duplicate a key. Insert must treat tombstones as writable, lookup must not stop on them.

Homework - due Friday 24 October, 17:00

Implement a chain-hash map in the language of the course. Keys are strings. Values are integers.

Required operations: insert, get, remove, and size. Grow the table when the load factor exceeds 0.75.

Submit a zip to the course portal before Friday 24 October, 17:00. Late work is not accepted this week. Put your name and student id in a README.

This is the only graded work from this lecture. The appendix is not assessed.

Appendix - optional reading (not assessed)

Knuth, The Art of Computer Programming, vol. 3, the section on hashing. Dense. Skip it unless you already finished the homework.

Office hours: Tuesday 14:00, room 4.12. Bring a drawing of your table, not a stack trace.

A longer list of string hash functions (djb2, sdbm, FNV-1a) and a table of prime moduli sits on the course site. None of it is in the exam.

Last year's quiz on binary search trees is also uploaded. It is the wrong unit. Do not study it this week.